

EEE3096S Practical 1A

STM32 GPIO, Timers, and Interrupts

Department of Electrical Engineering
University of Cape Town

2026

1 Overview

This practical recaps your STM32 knowledge from previous work. You will interface with the LEDs and pushbuttons on the developer board. You will extend your knowledge to timers and timer interrupts. Timers are a key feature of embedded systems. They help with timekeeping and scheduling operations. In this practical you will create delays with a timer. You will invoke a timer interrupt to run code at regular intervals. You will handle button inputs with debounce functionality. You will implement multiple LED flashing modes.

You will use STM32CubeIDE and the HAL libraries. These tools simplify programming by abstracting low-level hardware interactions. By the end of this practical you will have a deeper understanding of GPIO control and timer configuration. This practical is designed to resist AI-assisted cheating. You must make predictions before running your code. You must diagnose a deliberate hardware fault. You must take measurements with an oscilloscope. You must answer conceptual questions based on your specific implementation. Your measurements will vary between boards. Your answers cannot be copied from another student.

2 Outcomes and Knowledge Areas

After completing this practical you will know how to:

- Configure a timer to generate periodic interrupts.
- Write an interrupt service routine that responds to timer events.
- Implement software debounce for button inputs.
- Control LEDs in response to timer interrupts and button presses.
- Measure and verify timer timing behaviour using an oscilloscope.
- Diagnose and correct a configuration error in timer setup.

3 Deliverables

You must complete all tasks in the Tasks section. You must push your completed code to a shared repository on GitHub. You must demonstrate your working implementation to a tutor in the lab. You may conduct your demo during any lab session before the submission deadline. You must submit a single PDF file on Gradescope. Name your file with this format:

prac_01A_stdnum001(lowercase)_stdnum002(lowercase).pdf

Your practical mark is based on both your demo and your written submission. Both you and your partner receive the same mark.

4 Getting Started

Complete these steps before you begin coding.

1. Clone or download the Git repository. The practical folder of interest is Practical1/Practical1_A. Use this command:

```
git clone https://github.com/EEE3096S-UCT/EEE3096S-2026
```
2. Open STM32CubeIDE. Navigate through the menus: File, Import, Existing Code as Makefile Project. Click Next. In the dialog, click Browse and select your project folder. Under Toolchain, choose MCU ARM GCC. Click Finish.
3. The IDE provides a GUI to set up clocks and peripherals. This setup is stored in an .ioc file. We have provided this file in the project folder. You may look at it to see how pins are configured. Do not make or save any changes to this .ioc file. It would re-generate the code in your main.c file and may delete code you have added.
4. Open the main.c file under the Core/src folder. Complete the tasks below. All code you need to write is marked with a TODO comment. Do not edit any other provided code.

5 Hardware Orientation

Before you write any code, identify the following components on your board. Locate the physical pins. Record your answers. You will need them for later tasks. Find the eight user LEDs. Which GPIO pins connect to these LEDs? Find the four push buttons. Which GPIO pins connect to these buttons? Find the 8 MHz crystal on your board. Which pins does it connect to? What is the default system clock frequency when using the internal HSI oscillator? What is the purpose of the loopback header P2?

6 Background

6.1 GPIO Configuration

The STM32 HAL library provides functions to configure GPIO pins. Each pin can be set to input mode, output mode, alternate function mode, or analog mode.

For output pins, use these functions:

- `HAL_GPIO_WritePin(GPIO_Port, GPIO_Pin, GPIO_PIN_SET);`
- `HAL_GPIO_WritePin(GPIO_Port, GPIO_Pin, GPIO_PIN_RESET);`

For input pins, read the state with:

- `GPIO_PinState state = HAL_GPIO_ReadPin(GPIO_Port, GPIO_Pin);`
- The HAL handles the underlying register writes to `GPIOx_ODR`, `GPIOx_IDR`, and `GPIOx_MODER`.

6.2 Timer Fundamentals

A timer consists of three main components. The clock source provides the timer with a clock from the system clock. The system clock is divided by the prescaler. The prescaler divides the timer clock frequency. The timer counter increments at this rate:

$$\text{timer clock} = \frac{\text{system clock}}{\text{prescaler} + 1}$$

The auto-reload register sets the counter maximum. The counter counts from 0 up to the ARR value. When it reaches ARR, it resets to 0 and generates an update event. The update frequency is:

$$\text{Update Frequency} = \frac{\text{Timer Clock}}{\text{ARR} + 1}$$

Given:

System Clock = 8 MHz, Prescaler = 7999, ARR = 999

The timer clock is:

$$\text{Timer Clock} = \frac{8\,000\,000}{7999 + 1} = \frac{8\,000\,000}{8000} = 1000 \text{ Hz}$$

The update frequency is:

$$\text{Update Frequency} = \frac{1000}{999 + 1} = \frac{1000}{1000} = 1 \text{ Hz}$$

The timer period is:

$$\text{Period} = \frac{(\text{Prescaler} + 1)(\text{ARR} + 1)}{\text{System Clock}}$$

Substituting the values:

$$\text{Period} = \frac{(7999 + 1)(999 + 1)}{8\,000\,000} = \frac{8000 \times 1000}{8\,000\,000} = 1 \text{ s}$$

6.3 Timer Polling versus Timer Interrupt

Polling means you check the timer status flag in your main loop. You use `_HAL_TIM_GET_FLAG` and `_HAL_TIM_CLEAR_FLAG`. Polling blocks the processor if you wait for the flag. It may miss events if you check infrequently.

Interrupt means the timer triggers an interrupt on each update event. The processor stops executing the main loop. It runs the ISR. Then it returns to the main loop. This approach is more efficient and reliable.

6.4 Button Debounce

Mechanical switches bounce when pressed or released. A single press can appear as multiple transitions to a microcontroller. The solution is to ignore changes that occur within a short time window. This window is typically 10 to 50 ms after the first detected change.

Software debounce uses a timer or the `HAL_GetTick` function. You measure the time since the last detected press. You ignore any new press until the debounce time has elapsed.

7 Tasks

Complete the following tasks in order. Each task builds on the previous one. Tutors will verify your progress at checkpoints.

7.1 Task 1: Timer Configuration and Verification

Your goal is to configure TIM16 to generate an interrupt at 1 Hz and verify the timer frequency using an oscilloscope. You must calculate the prescaler and ARR values yourself, then diagnose and correct a deliberate configuration error. The system uses the HSI oscillator by default. The HSI frequency is 8 MHz.

Before you write any code or calculate any values, record your predictions. Write down the system clock frequency. Write down the prescaler value you expect to use. Write down the ARR value you expect to use. Write down the expected interrupt period in seconds.

Use this formula:

$$\text{Update Frequency} = \frac{\text{System Clock}}{(\text{Prescaler} + 1) * (\text{ARR} + 1)}$$

You want an update frequency of 1 Hz. Choose a prescaler value that divides the 8 MHz clock to a convenient frequency. Then set ARR to achieve the 1 Hz update. Show your working. Include the calculation for the timer clock frequency. Include the calculation for the ARR value.

Open the MX_TIM16_Init function in main.c. The code template contains blanks for the prescaler and period:

```
1 htim16.Init.Prescaler = _____;  
2 htim16.Init.Period = _____;
```

Connect the oscilloscope probe to PB0. Measure the toggle frequency. The toggle period equals twice the interrupt period because the pin changes state on every interrupt. Record the measured period. Compare this with your expected period. The code you have written should produce a 1 Hz interrupt.

7.2 Task 2: LED Control with Timer Interrupt

Your goal is to use a timer interrupt to control a continuous running light pattern. The sequence should illuminate one LED at a time, moving from left to right (LED0 to LED7), and then reverse direction.

To ensure the main loop remains non-blocking, use the TIM16_IRQHandler to set a global flag rather than manipulating the GPIO pins directly within the ISR. In your main loop, poll this flag to trigger the LED pattern update and clear it once handled. When changing direction, ensure the turn-around LEDs (LED0 and LED7) are not duplicated in the sequence. The illuminated indices should follow this pattern:

0, 1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1, 0, 1, 2, ...

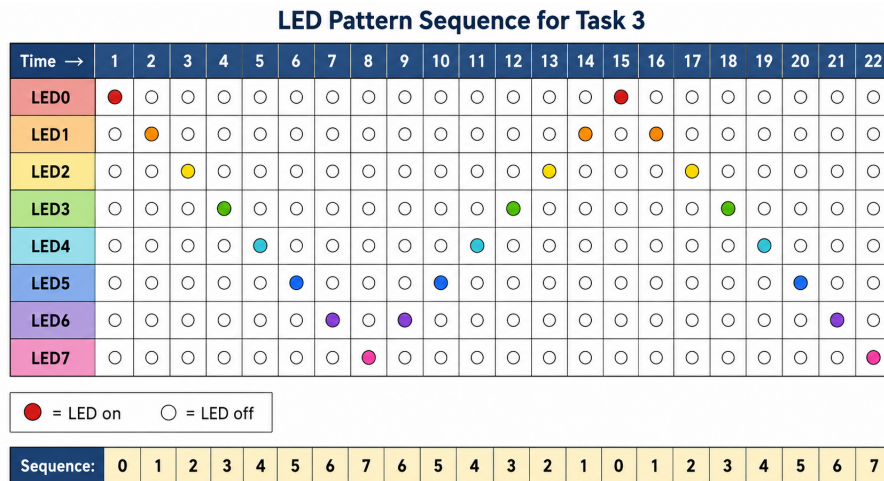


Figure 1: LED Sequence

Complete the provided C code skeleton. Utilize the pre-defined arrays that map GPIO ports and pins to each LED index. You must implement the logic to update the current LED state, control the directional flow, and ensure smooth transitions.

Submit your complete code for the interrupt handler and pattern update. Record a short video demonstrating the running light sequence operating correctly.

7.3 Task 3: Button Input and Software Debounce

Goal: Measure mechanical switch bounce. Implement software debounce. Toggle the timer speed between 1s and 0.5s using PA0.

Method: Connect the oscilloscope probe to PA0. Configure the oscilloscope for a single-shot trigger. Press the physical button. Capture the switch contact bounce waveform. Measure the total bounce duration. Write software debounce logic using `HAL_GetTick()`. Program the logic to ignore inputs during this specific bounce window.

Implementation: Your debounce logic must reliably toggle the timer speed without double-triggering. Alter the timer ARR register dynamically during runtime. Keep the prescaler fixed at 7999. Do not re-initialize the entire timer. Modify `TIM16->ARR` directly. Reset `TIM16->CNT` to zero.

Verification: Ensure the running light pattern speeds up or slows down exactly once per button press.

Record:

- Your expected switch bounce duration.
- The maximum bounce duration measured on the oscilloscope screen.
- Your chosen software debounce delay time.
- Your mathematical formula to calculate the ARR value from the desired period in milliseconds.

Question 4: When you update the ARR register dynamically, does the timer frequency change immediately mid-cycle, or does the timer wait for the next update event? Base your answer on the STM32 timer preload register behavior.

7.4 Task 4: Multi-Mode LED Control

Your goal is to combine all previous tasks into a complete system with three LED modes. All four buttons must be debounced.

- PA0 toggles the timer speed between 1 second and 0.5 second. This works in all modes.
- **Mode 1** is the running light pattern from Task 3. Press PA1 to select Mode 1.

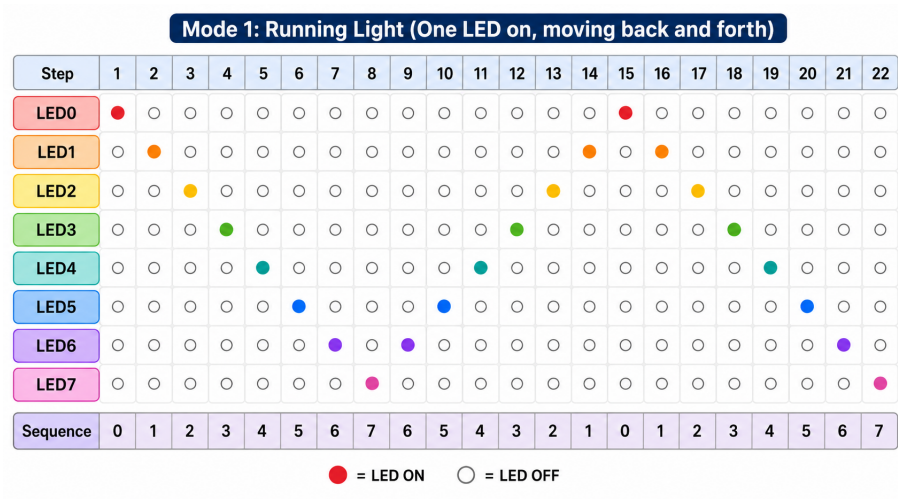


Figure 2: Mode 1

- **Mode 2** is the inverse running light pattern. All LEDs are on except one. The off LED moves 0 to 7 to 0. Press PA2 to select Mode 2.

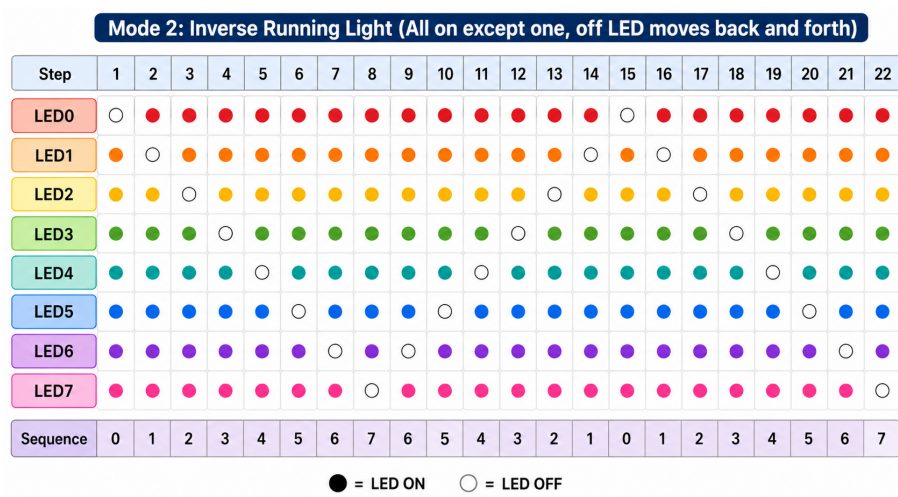


Figure 3: Mode 2

- **Mode 3** is the sparkle mode. Generate a random 8-bit pattern from 0 to 255. Display this pattern on the LEDs. Hold the pattern for a random duration between 100 ms and 1500 ms. Then turn off each LED one at a time. Use a random delay between 100 ms and 150 ms between each turn-off. After all LEDs are off, repeat with a new random pattern. Press PA3 to select Mode 3.

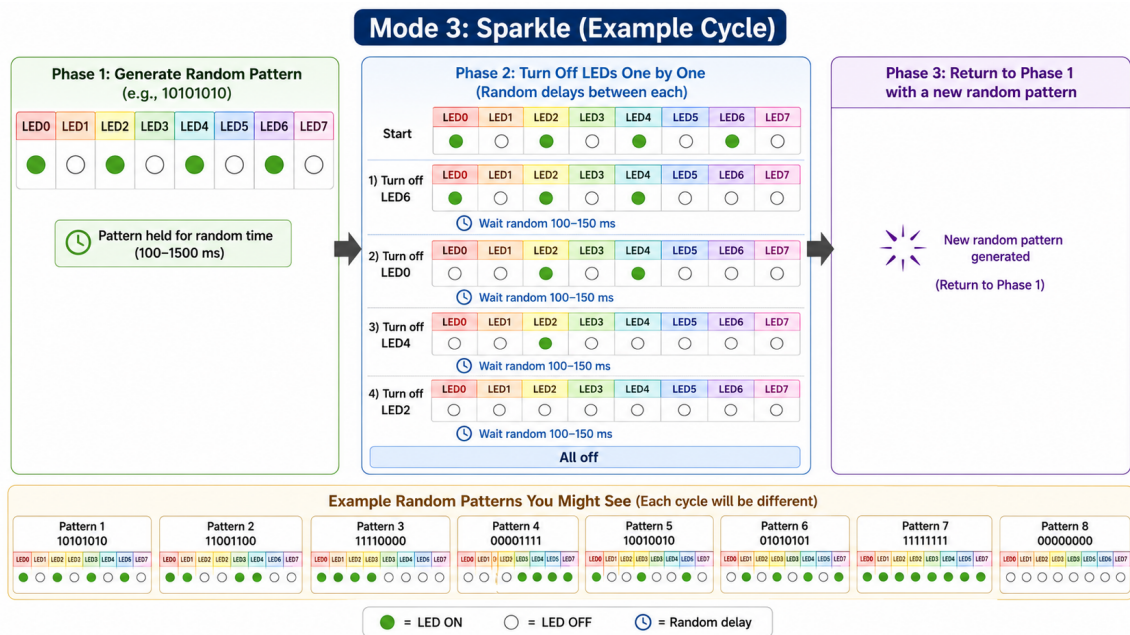


Figure 4: Mode 3

All timer updates must occur in the main loop. The ISR must only set the timer_event flag. The main loop handles all mode switching and pattern updates.

Mode changes must clear all LEDs before starting the new mode. Use the turn_off_all_leds function.

Implement a state machine for Mode 3. Use these states: SPARKLE_IDLE, SPARKLE_DISPLAY, and SPARKLE_TURN_OFF.

In SPARKLE_IDLE, generate a random pattern. Turn on the LEDs according to the pattern. Store which LEDs are on. Set a random display duration. Transition to SPARKLE_DISPLAY.

In SPARKLE_DISPLAY, wait for the display duration to elapse. Use HAL_GetTick to check the time. When the time expires, transition to SPARKLE_TURN_OFF.

In SPARKLE_TURN_OFF, turn off one LED at a time. Use random delays between each turn-off. When all LEDs are off, transition back to SPARKLE_IDLE.

Submit your complete main.c file. Submit a written explanation of your state machine implementation for Mode 3. Record and submit the measured minimum and maximum LED-on times for Mode 3. Record and submit a video demonstration showing all three modes and the speed toggle.

7.5 Task 5: ADC Interrupts and LED Level Meter

Goal: Read a variable DC voltage from an external source using the microcontroller's Analog-to-Digital Converter (ADC). Display this voltage magnitude dynamically as a "bar graph" on your 8-bit LED array using ADC interrupts.

Method: Configure the signal generator to output a pure DC voltage (use the DC offset function with 0 V amplitude). Connect this DC output to pin PA5 on the expansion header. Configure PA5 as an analog input. Set up the ADC to run in continuous conversion mode and trigger an interrupt at the end of each conversion. In the ADC interrupt service routine, read the 12-bit ADC value and map it proportionally to the 8 LEDs (e.g., 0 V = 0 LEDs on, 3.3 V = 8 LEDs on).

Verification: Use the digital multimeter to measure the exact DC voltage present at the PA5 pin. Slowly increase the DC offset on the signal generator and observe the LEDs.

Record:

- Your predicted 12-bit ADC register value (in decimal) for an input of exactly 2.0 V.
- The actual measured multimeter voltage at the exact moment the 5th LED turns on.
- A calculation showing whether your measured voltage for the 5th LED matches the theoretical voltage threshold for that step.

Question: If a student accidentally sets the signal generator to output a 5.0 V DC signal into PA5, what physical damage might occur? Consult the “Pinouts and pin description” table in the STM32F051x datasheet. Look at the “I/O structure” column for PA5 and explain what that specific abbreviation means regarding voltage tolerance.

7.6 Task 6: Software PWM via Timer Interrupts

Goal: Generate a 100 Hz custom Pulse Width Modulation (PWM) signal with a 30% duty cycle on pin PB4. You must use a basic timer update interrupt and standard GPIO control. You must not use any hardware PWM peripheral features.

Method: Configure TIM16 to generate an interrupt at a high frequency of 10 kHz. Inside the timer interrupt service routine, create a static counter variable. Increment this counter from 0 to 99, then reset it to 0. If the counter is less than 30, drive PB4 logic high. If the counter is 30 or greater, drive PB4 logic low.

Verification: Connect the oscilloscope probe to PB4. Adjust the timebase to view two complete periods of the signal. Measure the total period, the frequency, and the positive pulse width.

Record:

- Your calculated Timer Prescaler and ARR values to achieve exactly 10 kHz.
- The measured total period of the signal from the oscilloscope screen.
- The measured positive pulse width from the oscilloscope screen.

Question: A project requires you to generate a 50 Hz signal with a duty cycle resolution of 1 percent. What is the exact timer interrupt frequency required to achieve this timing constraint? Show your mathematical calculation.